Deep Hooks: Monitoring native execution in WoW64 applications – Part 3

March 28, 2018 (https://www.sentinelone.com/blog/2018/03/)
by Yarden Shafir & Assaf Carlsbad (https://www.sentinelone.com/blog/author/carlsbad/)

Introduction

Last time (part 1 (https://www.sentinelone.com/blog/deep-hooks-monitoring-native-execution-wow64-applications-part-1/), part 2 (https://www.sentinelone.com/blog/deep-hooks-monitoring-native-execution-wow64-applications-part-2/)) we demonstrated several different methods for injecting 64-bit modules into WoW64 (https://www.sentinelone.com/blog/now-stage-deep-hooks-monitoring-native-execution-wow64-applications/) processes. This post will pick up where we left off and describe how the ability to execute 64-bit code in such processes can be leveraged to hook native x64 APIs. To accomplish this task, the injected DLL must host a hooking engine capable of operating in the native region of a WoW64

process. Unfortunately, none of the hooking engines we inspected can manage this out-of-the-box, so we were forced into modifying one of them in order to make it suit our needs.

Creating the injected DLL

Choosing the hooking engine to adapt

Hooking is a well-established technique in the field of computer security, used extensively by defenders (https://quequero.org/2014/10/kaspersky-hooking-engine-analysis/) and attackers (https://www.sans.org/reading-room/whitepapers/forensics/analyzing-man-in-the-browser-mitb-attacks-35687) alike. Since the publication of the seminal article *Detours: Binary Interception of Win32 Functions* (https://www.cs.columbia.edu/~junfeng/10fa-e6998/papers/detours.pdf) back in 1999, many different hooking libraries were developed. Most of them exercise similar concepts to the ones presented in that article, but differ in other aspects such as their support for various CPU architectures, support for transactions, etc. Out of these libraries, we had to choose one which best fit our requirements:

1. Support inline hooking of x64 functions.
2. Open-sourced and free-licensed – so that we could modify it legally.
3. Preferably, the hooking engine should be relatively minimal so as to require as few modifications as possible.

After taking all of these requirements into consideration, we chose MinHook (https://github.com/TsudaKageyu/minhook) as our go-to engine. What eventually tipped the scales in favor of it was its small codebase, making it relatively easy to use in a PoC. All of the modifications presented later were done on top of it, and might be slightly different if another hooking engine is used instead.

The complete source code of our modified hooking engine is available **here** (https://github.com/Sentinel-One/minhook).

Look ma', no dependencies!

In part 1 we briefly mentioned that not any 64-bit module can be easily loaded into a WoW64 process. Most DLLs tend to use (both implicitly and explicitly) various functions found in common Win32 subsystem DLLs, such as *kernel32.dll*, *user32.dll*, etc. However, the 64-bit versions of these modules are not loaded by default into WoW64 processes, as they are not required for the WoW64 subsystem to operate. Furthermore, due to some limitations imposed by the address space layout, forcing the process to load any of these is somewhat difficult and unreliable.

To avoid unnecessary complications, we chose to modify our hooking engine and the DLL that hosts it so that they would only rely on native 64-bit modules found normally in WoW64 processes. Basically, this left us with just the native NTDLL, as the DLLs comprising the WoW64 environment don't usually contain functions which are beneficial to us.

In a more practical sense, to force the build environment to only link against NTDLL, we specify the */NODEFAULTLIB* flag in the linker settings, and explicitly add *"ntdll.lib"* to the list of additional dependencies:

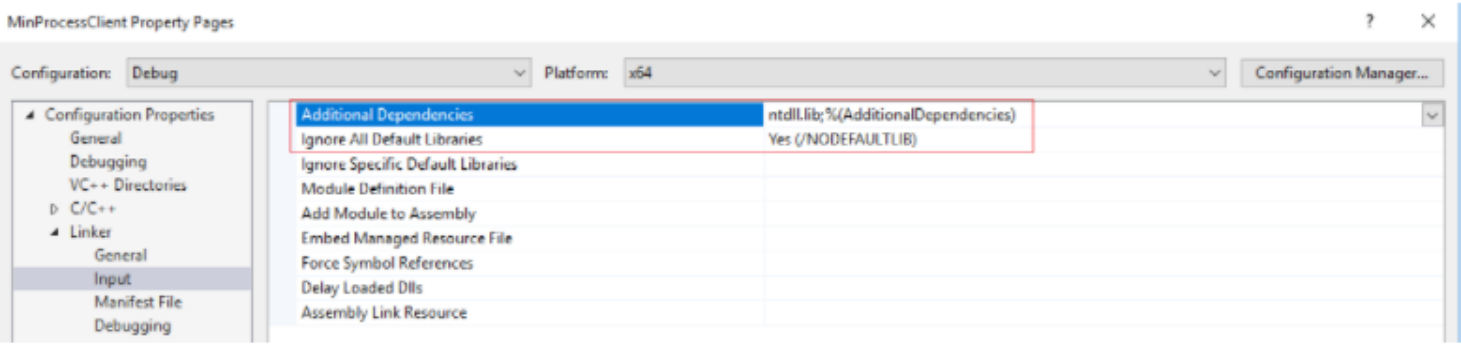


Figure 19 – linker configuration for the host DLL

API re-implementation

The first and most noticeable effect that this change raises is that higher-level Win32 API functions are not available at our disposal and would have to be re-implemented using their NTDLL counterparts. As demonstrated in figure 20, for every Win32 API used by MinHook, we introduced a replacement function which has the same public interface and implements the same core functionality, while internally using only NTDLL facilities.

Most of the time these "translations" were rather straightforward (for example calls to *VirtualProtect()* can be replaced almost directly with calls to *NtProtectVirtualMemory()*). In other, more complex cases the mapping between the

Win32 API functions and their native counterparts is not as clear, so we had to resort to some reverse engineering or peeking inside the ReactOS sources (https://github.com/reactos/reactos).

```
BOOL WINAPI MyVirtualProtect(
    _In_  LPVOID lpAddress,
    _In_  SIZE_T dwSize,
    _In_  DWORD  flNewProtect,
    _Out_ PDWORD lpflOldProtect
)
{
    NTSTATUS Status;
    Status = NtProtectVirtualMemory(NtCurrentProcess(),
                                    &lpAddress,
                                    &dwSize,
                                    flNewProtect,
                                    (PULONG)lpflOldProtect);
    if (!NT_SUCCESS(Status)) {
        return FALSE;
    }
    return TRUE;
}
```

Figure 20 – private implementation of VirtualProtect()

Project configuration

After re-implementing all Win32 API calls in MinHook, we were still left with a bunch of errors:

```
LNK2001 unresolved external symbol _DllMainCRTStartup
LNK2001 unresolved external symbol memcpy
LNK2001 unresolved external symbol _RTC_CheckStackVars

LNK2001 unresolved external symbol _RTC_InitBase
LNK2001 unresolved external symbol _RTC_Shutdown
LNK2001 unresolved external symbol __GSHandlerCheck
LNK2001 unresolved external symbol __security_check_cookie

LNK2001 unresolved external symbol __security_cookie
LNK2019 unresolved external symbol memcpy referenced in function MH_CreateHook
LNK2019 unresolved external symbol _RTC_CheckStackVars referenced in function MH_CreateHook
LNK2019 unresolved external symbol _RTC_CheckStackVars referenced in function MH_CreateHook
LNK2019 unresolved external symbol __security_check_cookie referenced in function MH_CreateHook
LNK2019 unresolved external symbol __security_check_cookie referenced in function MH_CreateHook
LNK2019 unresolved external symbol __security_cookie referenced in function MH_CreateHook
LNK2019 unresolved external symbol __security_cookie referenced in function MH_CreateHook
LNK2019 unresolved external symbol memset referenced in function AllocateBuffer
LNK2019 unresolved external symbol memset referenced in function AllocateBuffer
LNK2019 unresolved external symbol _RTC_UninitUse referenced in function hde64_disasm
```

Figure 21 – a bunch of errors

Luckily, solving most of these errors only requires slight configuration changes to the project. As can be seen in the figure, most of the errors take the form of an unresolved external symbol normally exported from the CRT (which is not available). They can be settled by changing a few flags in the linker settings:

- Disable basic runtime checks (remove the */RTC* flag from the command line)
- Disable buffer security checks (*/GS-* flag)

- The entry point must be explicitly specified as *DllMain* since *DllMainCRTStartup* is not linked in.
- Additionally, *memcpy()* and *memset()* have to be implemented manually, or replaced with calls to *RtlCopyMemory()* and *RtlFillMemory()* exported from NTDLL.

After applying all of these changes, we successfully created a custom 64-bit DLL that contains no dependencies other than NTDLL:

CFF Explorer VII - [InjectedDll.dll]

File Settings ?

- File: InjectedDll.dll
  - Dos Header
  - Nt Headers
    - File Header
    - Optional Header
      - Data Directories [x]
  - Section Headers [x]
  - Import Directory
  - Resource Directory
  - Relocation Directory
  - Debug Directory
  - Address Converter
  - Dependency Walker
  - Hex Editor
  - Identifier
  - Import Adder
  - Quick Disassembler
  - Rebuilder
  - Resource Editor

InjectedDll.dll

Module Name	Imports	OFTs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
0000AAA8	N/A	0000A920	0000A924	0000A928	0000A92C	0000A930
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
ntdll.dll	21	0000E148	00000000	00000000	0000E2A8	0000E000

OFTs	FTs (IAT)	Hint	Name
Qword	Qword	Word	szAnsi
000000000000E268	000000000000E268	01C6	NtQueryInformationProcess
000000000000E284	000000000000E284	003C	EtwEventWrite
000000000000E294	000000000000E294	0039	EtwEventRegister
000000000000E2B2	000000000000E2B2	015A	NtGetContextThread
000000000000E2C8	000000000000E2C8	021F	NtSetContextThread
000000000000E2DE	000000000000E2DE	01B1	NtProtectVirtualMemory
000000000000E2F8	000000000000E2F8	014E	NtFlushInstructionCache
000000000000E312	000000000000E312	0154	NtFreeVirtualMemory
000000000000E328	000000000000E328	020F	NtResumeThread
000000000000E33A	000000000000E33A	01E8	NtQueryVirtualMemory
000000000000E352	000000000000E352	00CC	NtAllocateVirtualMemory
000000000000E36C	000000000000E36C	0258	NtSuspendThread
000000000000E37E	000000000000E37E	0427	RtlInitAnsiString
000000000000E392	000000000000E392	0432	RtlInitUnicodeString
000000000000E3AA	000000000000E3AA	0421	RtlImageNtHeader
000000000000E3BE	000000000000E3BE	0072	LdrGetDllHandle
000000000000E3D0	000000000000E3D0	007A	LdrGetProcedureAddress
000000000000E3EA	000000000000E3EA	0401	RtlGetProcessHeaps
000000000000E400	000000000000E400	02BB	RtlAllocateHeap
000000000000E412	000000000000E412	03C4	RtlFreeHeap
000000000000E420	000000000000E420	01C8	NtQueryInformationThread

Figure 22 – a minimalistic DLL with only a single import descriptor (NTDLL)

Hooking the native NTDLL

Once we modified our hooking engine to match all the aforementioned limitations, we can take a closer look at the hooking mechanism itself. The hooking technique employed by MinHook, as well as by the vast majority of such libraries, is dubbed "inline hooking (https://www.malwaretech.com/2015/01/inline-hooking-for-programmers-part-1.html)". The inner workings of this technique are rather well-documented, but here is a simplified description of the steps this method includes:

1. Allocate a "trampoline" in the process' address-space and copy into it the prolog of the function that will eventually be hooked.
2. Place a JMP instruction in the trampoline, right after the copied prolog. This JMP should point back to the instruction following the prolog of the original function.
3. Place another JMP instruction in the trampoline, right before the copied prolog. This JMP should point into a detour function (usually found in the DLL we previously injected into the process).
4. Overwrite the hooked function prolog with a JMP instruction pointing to the trampoline.

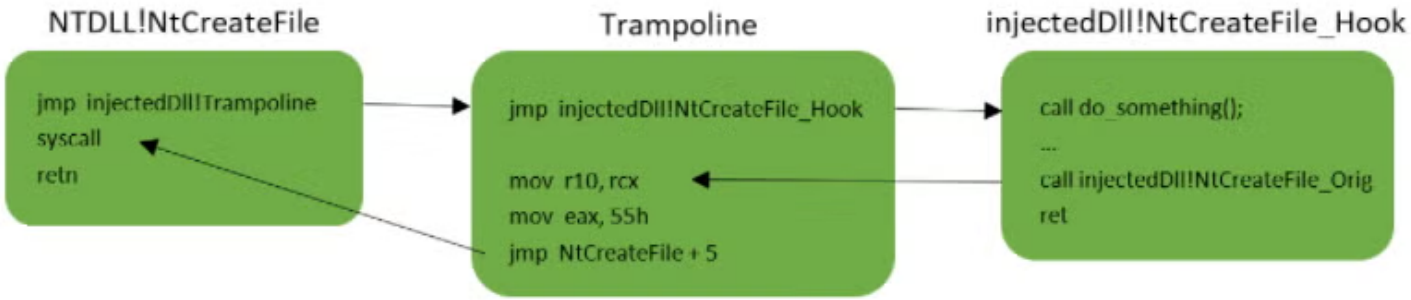


Figure 23.1 – a general illustration of what an inline hook looks like.

```
0:009> u 00007ff9`e1ca0178
00007ff9`e1ca0178 ff25f2ffffff    jmp     qword ptr [00007ff9`e1ca0170]
00007ff9`e1ca017e cc              int     3
00007ff9`e1ca017f cc              int     3
00007ff9`e1ca0180 4c8bd1          mov     r10,rcx
00007ff9`e1ca0183 b81e000000      mov     eax,1Eh
00007ff9`e1ca0188 ff253a000000    jmp     qword ptr [00007ff9`e1ca01c8]
```

Figure 23.2 – a view of the trampoline. Marked in red is the jump to the detour function, and marked in green are the instructions copied from the hooked function and the jump back into it.

This hooking method works by modifying the prolog of the hooked function, so whenever it is called by the application, the detour function will be called instead. The detour function can then execute any code before, after or instead of the original function.

In 64-bit mode, most hooking engines use two different types of jumps for the hooked function and the trampoline:

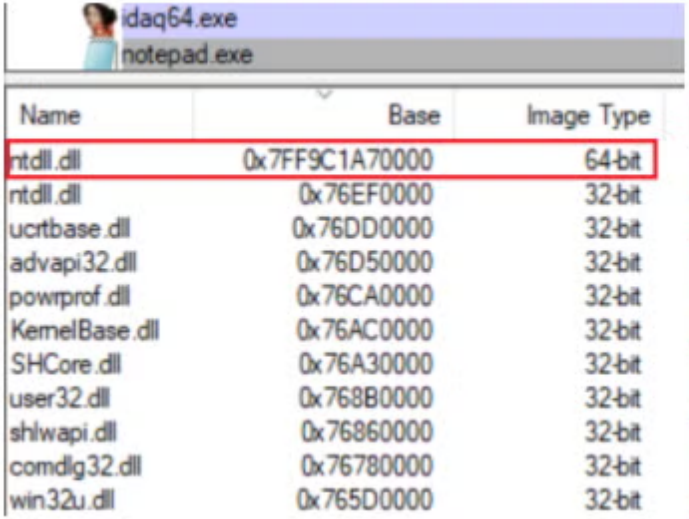
1. The jump from the hooked function to the trampoline is a relative jump (https://c9x.me/x86/html/file_module_x86_id_147.html) encoded as *"E9 <4 byte offset>"*. Since this instruction operates on a DWORD-sized operand, the trampoline must be at a distance of no more than 2GB away from the hooked

function. This form of jump is usually chosen for this step since it only takes up 5 bytes and so it's compact enough to fit neatly inside the function's prolog.

2. The jumps from the trampoline into the detour function and back to the hooked function, shown in figure 23.2, are indirect, RIP-relative jumps encoded as *"FF25 <4 byte offset>"* (mnemonic form: *JMP qword ptr [rip+offset]*). This instruction will jump into a 64-bit absolute address stored in the location pointed to by RIP plus the offset.

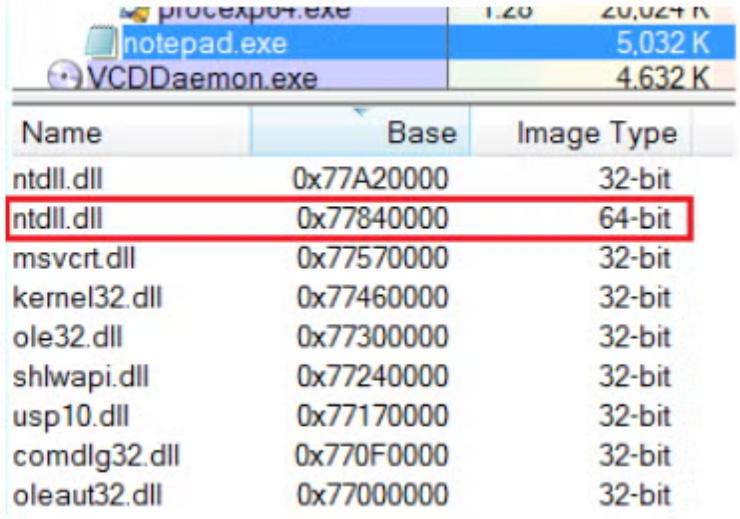
When running in native 64-bit processes, hooking engines employing this technique work just fine. As can be expected, the trampoline is allocated a short distance from the target function (up to 2GB away), thus allowing for successful binary instrumentation.

However, some recent changes to the memory layout of WoW64 processes guarantee that this technique cannot be applied to the native NTDLL without some additional changes. As Alex Ionescu demonstrated in his blog (http://www.alex-ionescu.com/?p=300), in recent Windows versions (starting from Windows 8.1 update 3), the native NTDLL has been relocated: instead of being loaded into the lower 4GB of the address space together with the rest of the process' modules, it is now loaded to a much higher address.



idaq64.exe
notepad.exe

Name	Base	Image Type
ntdll.dll	0x7FF9C1A70000	64-bit
ntdll.dll	0x76EF0000	32-bit
ucrtbase.dll	0x76DD0000	32-bit
advapi32.dll	0x76D50000	32-bit
powrprof.dll	0x76CA0000	32-bit
KernelBase.dll	0x76AC0000	32-bit
SHCore.dll	0x76A30000	32-bit
user32.dll	0x768B0000	32-bit
shlwapi.dll	0x76860000	32-bit
comdlg32.dll	0x76780000	32-bit
win32u.dll	0x765D0000	32-bit



notepad.exe 5,032 K
VCDDaemon.exe 4,632 K

Name	Base	Image Type
ntdll.dll	0x77A20000	32-bit
ntdll.dll	0x77840000	64-bit
msvcrt.dll	0x77570000	32-bit
kernel32.dll	0x77460000	32-bit
ole32.dll	0x77300000	32-bit
shlwapi.dll	0x77240000	32-bit
usp10.dll	0x77170000	32-bit
comdlg32.dll	0x770F0000	32-bit
oleaut32.dll	0x77000000	32-bit

Figure 24 – base address of 64-bit NTDLL on Windows 10 (left) and Windows 7 (right).

The rest of the address space above the 4GB boundary (with the exception of the native NTDLL and the native CFG bitmap) is protected with a SEC_NO_CHANGE VAD and thus cannot be accessed, allocated or freed by anyone. This means that the trampoline will always be allocated in the lower 4GB of the address space. Since the total user-mode address space in 64-bit systems is 128TB, the distance between the native NTDLL and the trampoline is bound to be much greater than 2GB. That makes the JMP emitted by most hooking engines inadequate.

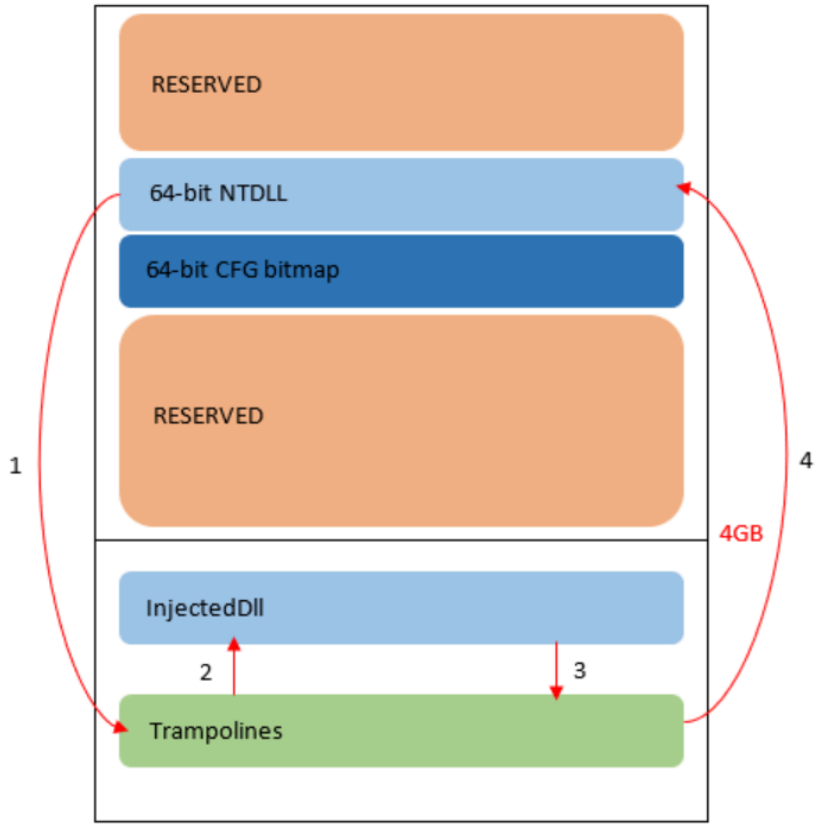


Figure 25 – illustration showing the control transfers required for inline hooks in WoW64 processes on Windows 8.1 and up. Note that in Windows 10 RS4 preview (build 17115) the SEC_NO_CHANGE VADs don't seem to exist anymore (https://twitter.com/yarden_shafir/status/973312135160909824) and memory can be allocated anywhere in the process address space.

An alternative form of JMP

To overcome this issue, we had to replace the relative JMP with a different instruction, capable of passing a distance of up to 128TB. When searching for alternatives, we bumped into a post by Gil Dabah listing a few possible options

(http://www.ragestorm.net/blogs/?p=107). After disqualifying every option that "dirties" a register, we were left with only a couple of viable choices. Initially, we attempted to replace the relative JMP with an indirect, RIP-relative JMP similar to the ones used in the trampoline:

```
JMP [RIP+0]
<address of trampoline>
```

This instruction performed well on Windows 10, and provided us with a way to instrument various native API functions in WoW64 processes. But when testing the modified code on earlier Windows versions such as Windows 8.1 and Windows 7, it failed to create the hooks entirely. As it turns out, NTDLL functions in these Windows versions are shorter than their counterparts in Windows 10 (http://blog.amossys.fr/windows10_TH2_int2E_mystery.html), and usually do not contain enough space to accommodate our chosen JMP instruction, which takes up 14 bytes.

```
public ZwAllocateVirtualMemory
ZwAllocateVirtualMemory proc near
mov     r10, rcx
mov     eax, 18h
test    byte ptr ds:7FFE0308h, 1
jnz     short loc_1800A56A5
```

```
syscall
retn
```

```
loc_1800A56A5:
int     2Eh
retn
ZwAllocateVirtualMemory endp
```

```
public ZwAllocateVirtualMemory
ZwAllocateVirtualMemory proc near
mov     r10, rcx
mov     eax, 17h
syscall
retn
ZwAllocateVirtualMemory endp
```

Figure 26 – implementation of ZwAllocateVirtualMemory() in Windows 10 RS2 (left) and Windows 8.1 (right).

To make our DLL universal across all Windows versions, we had to find a shorter instruction, still capable of branching to the trampoline. Eventually, we came up with a solution that actually takes advantage of the trampoline's location: since the trampoline must be allocated in the lower 4GB of the address space, the upper 4 bytes of its 8-byte address are zeroed out. This lets us use the following option, which only takes up 6 bytes:

```
PUSH <4 byte address>
RET
```

The reason this method works is because in x64 code the PUSH instruction, when supplied with a 4-byte operand, actually pushes an 8-byte value onto the stack. The upper 4 bytes are used as a sign extension, meaning that as long as the 4-byte address is not greater than 2GB, they will be zeroed.

We then use a RET instruction, which pops an 8 byte address from the stack and jumps to it. Since we have just pushed the address of the trampoline to the top of the stack, that would be our return address.

```
0:000> u ntdll!NtAllocateVirtualMemory
ntdll!NtAllocateVirtualMemory:
00007ffa`21cb0160 6878019200      push    920178h
00007ffa`21cb0165 c3              ret
00007ffa`21cb0166 cc              int     3
00007ffa`21cb0167 cc              int     3
00007ffa`21cb0168 f604250803fe7f01 test    byte ptr [SharedUserData+0x308 (00000000`7ffe0308)],1
00007ffa`21cb0170 7503            jne     ntdll!NtAllocateVirtualMemory+0x15 (00007ffa`21cb0175)
00007ffa`21cb0172 0f05            syscall
00007ffa`21cb0174 c3              ret
```

Figure 27 – NtAllocateVirtualMemory() containing our modified hook. Notice the first two instructions, which push the address of the trampoline into the stack and immediately "return" to it.

There is only one problem left with this method, caused by CFG. As mentioned in part 2 of this series (https://www.sentinelone.com/blog/deep-hooks-monitoring-native-execution-wow64-applications-part-2/), all private memory allocations in WoW64 processes – including the trampolines used for our hooks – are marked exclusively in the WoW64 CFG bitmap.

Whenever we wish to execute the original API function from the detour, we first need to call the trampoline in order to run that function's prolog. But, if our DLL is compiled with CFG, it will attempt to validate the trampoline's address against the native CFG bitmap before calling it. Due to this mismatch, the validation will fail, resulting in the termination of the process.

The solution to this problem is rather straightforward – having control over the DLL's configuration, we can simply compile it without enabling CFG. This is done by removing the */guard:cf* flag from the compiler's command line.

Preventing infinite recursions

The last issue to consider when adapting a hooking engine is infinite recursions. After the hooks are placed, whenever a call is made to a hooked function, this call will reach our detour instead. But our detour functions also execute their own code, which might

itself make calls to hooked functions, leading us back to our detour. Unless handled carefully, this can lead to infinite recursions.

```
0:000> k
 # Child-SP          RetAddr           Call Site
00 00000000`0097e318 00000000`771b1e0b InjectedDll!LdrLoadDll_Hook
01 00000000`0097e320 00000000`771b1e0b InjectedDll!LdrLoadDll_Hook+0x9b
02 00000000`0097e380 00000000`771b1e0b InjectedDll!LdrLoadDll_Hook+0x9b
03 00000000`0097e3e0 00000000`771b1e0b InjectedDll!LdrLoadDll_Hook+0x9b
04 00000000`0097e440 00000000`771b1e0b InjectedDll!LdrLoadDll_Hook+0x9b
05 00000000`0097e4a0 00000000`771b1e0b InjectedDll!LdrLoadDll_Hook+0x9b
06 00000000`0097e500 00000000`771b1e0b InjectedDll!LdrLoadDll_Hook+0x9b
07 00000000`0097e560 00000000`771b1e0b InjectedDll!LdrLoadDll_Hook+0x9b
08 00000000`0097e5c0 00000000`701cdc71 InjectedDll!LdrLoadDll_Hook+0x9b
09 00000000`0097e620 00000000`701cdb73 wow64!CpuLoadBinaryTranslator+0x8d
0a 00000000`0097e8b0 00000000`701cbec2 wow64!ProcessInit+0x1b7
0b 00000000`0097e8e0 00007ff9`c1b45c1d wow64!Wow64LdrpInitialize+0x142
0c 00000000`0097eb90 00007ff9`c1b3761b ntdll!LdrpInitializeProcess+0x18a1
0d 00000000`0097efd0 00007ff9`c1ae923b ntdll!_LdrpInitialize+0x4e3cb
0e 00000000`0097f050 00007ff9`c1ae91ee ntdll!LdrpInitialize+0x3b
0f 00000000`0097f080 00000000`00000000 ntdll!LdrInitializeThunk+0xe
```

Figure 28 – infinite recursions when the hook on LdrLoadDll attempts to load another DLL

Normally there exists a simple solution for this problem: declaring a thread local variable, which counts the “depth” of the recursion we’re in, and only executing the code inside the detour function the first time (counter == 1):

```
extern __declspec(thread) USHORT g_HookNestingLevel;

PVOID NTAPI Func_Hook()
{
    g_HookNestingLevel++;

    do {
        if (g_HookNestingLevel > 1) {
            break;
        }

        do_something();
    } while (false);

    g_HookNestingLevel--;

    PVOID Result = Func();
    return Result;
}
```

Figure 29 – a thread local variable counting the depth of the recursion

Unfortunately, we cannot use thread local variables in our DLL, for two reasons:

1. Implicit TLS (http://www.nynaeve.net/?p=183) (*__declspec(thread)*) relies heavily on the CRT, which is not available to us.
2. Explicit TLS (http://www.nynaeve.net/?p=181) APIs (*TlsAlloc() / TlsFree()*, etc.) are implemented in their entirely in kernel32.dll, whose 64-bit version is not loaded into WoW64 processes.

In spite of these constraints, *wow64.dll* does use TLS storage, as can be verified by looking at the output of the "*!wow64exts.info*" command:

```
Native TLS slots:

TLS_STACKPTR64:        0x0000000000000000
TLS_CPURESERVED:       0x000000000a29fd20
TLS_INCPUSIMULATION:   0x0000000000000000
TLS_LOCALTHREADHEAP:   0x0000000000000000
TLS_EXCEPTIONADDR:     0x0000000000000000
TLS_USERCALLBACKDATA:  0x0000000000000000
TLS_EXTENDED_FLOAT:    0x0000000000000000
TLS_APCLIST:           0x0000000000000000
TLS_FILESYSREDIR:      0x0000000000000000
TLS_LASTWOWCALL:       0x0000000000000000
TLS_WOW64INFO:         0x0000000002b2d468
```

Figure 30 – TLS variables used by the WoW64 DLLs

As it turns out, *wow64.dll* does not dynamically allocate TLS slots at runtime but rather uses hardcoded locations in the *TlsSlots* array accessible directly from the TEB (already instantiated on a per-thread basis).

```
loc_6B0065D9:
mov     rcx, gs:30h
mov     rax, [rsp+8B8h+var_880]
mov     [rcx+(NT_TEB.TlsSlots[11])], rax
mov     rax, cs:Wow64InfoPtr
mov     ecx, [rax+8]
test    ecx, ecx
jnz     loc_6B019802
```

Figure 31 – Wow64SystemServiceEx writes a thread local variable into a hardcoded location in the TlsSlots array

After some empirical testing, we discovered that most TLS slots in the 64-bit TEB are never used by WoW64 DLLs, so for the purpose of this PoC we can just pre-allocate one of them to store our counter. There is no guarantee that this slot will remain unused in future Windows versions, so production-grade solutions would probably look into some other available members of the TEB.

```
PTEB GetCurrentTeb64()
{
    static constexpr ULONG ThreadBasicInformation = 0;
    THREAD_BASIC_INFORMATION tbi{};
    NtQueryInformationThread(NtCurrentThread(), (THREADINFOCLASS)(ThreadBasicInformation),
&tbi, sizeof(tbi), nullptr);

    return (PTEB)(tbi.TebBaseAddress);
}
PULONG_PTR GetCurrentNestingLevelPtr()
{
    return (PULONG_PTR)(&GetCurrentTeb64()->TlsSlots[63]);
}
```

Figure 32 – using an unused member of the TEB to count the "depth" of our recursion.

Conclusion

This concludes the third and final part of our "Deep Hooks" series. In these three posts we presented several different ways to inject a 64-bit DLL into a WoW64 process and then use it to hook API functions in the 64-bit NTDLL. Hopefully, this option would benefit security products by allowing them to gain better visibility into WoW64 processes and making them more resilient to bypasses such as "Heaven's Gate".

The methods presented throughout this series still have their limitations, found in the form of new mitigation options (https://msdn.microsoft.com/en-us/library/windows/desktop/ms686880(v=vs.85).aspx) such as dynamic code

restrictions, CFG export suppression and code integrity guard. When enabled, these might prevent us from creating our hooks or thwart our injection altogether, but more on that in some future post.